Assignment - 7

Q1 find first occurance :-

expectation = target चा first index शोधायचा.
faith = recursion target चा index देईल.
base Condn = index == arr.length return 0.

arr [] = [5, 2, 3, 2, 4] target = 2

```
psvm(){
    int arr[] = {5,2,3,2,4}
    int val = fun (arr,0,2)
    Sop (val);
}
int fun (int arr [], int index, int target){
        if (arr.length == index){
                return -1;
        }
        if (arr[index] == target){
                return index;
        }
        return fun (arr, index+1, target);
}
```

Output = 1. 2 हा element first ocurance चा 1 index वर आहे.

2) find last ocurance :-

expectation :- target चा last चा index return करायचा.

faith - पुढचा recursion मध्ये last element साधेल.

base condn -; index == arr.length - return

2) find last ocurance :-
arr = [5, 2, 3, 2, 4]

```
psvm () {
    int arr [] = { 5, 2, 3, 2, 4 }.
    int val = fun (arr, 0, 2);
    Sop (val);
}
int fun( int arr [], int index , int target){
        if (arr.length == index) {
                return -1;
    }
    int right = fun (arr, index + 1 , target);
     if (right != -1){
                return ~~index~~ ;right;
    }
    ~~else {~~
            ~~return index~~
    if (arr [index] == target) {
                return index;
    }
    return -1;
}
```

Output :- 3

3) Count ocurrance :-

expectation - target किती वेळा येतो ते count करायच.

faith - array मधल बाकीच च Count देईल.

base Condition -

index == arr.length return 0.

```
psvm ( ){
    int arr[] = {1,2,2,3,2} target = 2.
    int val = fun (arr, 0, 2);
    Sop (val);
}
int fun (int arr[], int index, int target){
        if (arr.length == index){
                return 0;
        }
        int Count = fun (arr[], index+1, target);
        if (arr[index] == target){
                return Count + 1;
        }
        return Count;
}
```

Output :- 3.

4) check if array is Sorted :-

expectation - array हा ascending order मध्ये आहे का ते check करणे.

faith -: बाकीचा array sorted आहे का ते recursion करेल.

base cond -: index == arr.length - 1 return true

```
psum () {
    int arr [] = { 1,2,3,4 }.
    boolean val = fun ( arr, 0);
    Sop (val);
}
boolean fun (int arr [], int index) {
        if ( index == arr.length-1) {
                return ~~fal~~ true;
        }
    if (arr [index ] > arr [index+1]) {
            return false;
    }
    return fun (arr, index +1);
}
```

output :- true.

array मधला पहिला element हा array मधल्या दुसऱ्या element पेक्षा मोठा आहे याचा अर्थ तो sorted नाही म्हणून false return केला.

3] Check if all elements are even :-

expectation - array मधले सगळे elements even आहेत की नाय ते check करणे.
faith - बाकीचे elements recursion check करेल.
base condition = index == arr.length return true.

```
psvm () {
    int arr [] = {2,4,6,8}
    boolean val = fun (arr, 0);
    Sop (val);
}
boolean fun (int arr [], int index) {
    if (index == arr.length) {
        return true;
    }
    if (arr [index] % 2 != 0) {
        return false;
    }
    return fun (arr, index +1);
}
```

Output : true.

आपला array मधल्या element ला 2ने mod केल जर remainder 0 येत नसेल म्हणजे त्यातला element हा odd आहे. म्हणून false return करते.

6) check if any element is negative :—

expectation — array मध्ये 1 तरी element ला negative आले का

faith — बाकीच्या array मध्ये negative no असेल तर recursion सांभाळेल.

base case — index == arr.length return false.

```
psvm () {
    int arr [] = { 1, 2, -3, 4 }
    boolean val = fun (arr, 0);
    Sop (val);
}

    boolean fun (int arr [], int index) {
        if (index == arr.length) {
            return false;
        }
        if (arr [index] < 0) {
            return true;
        }
        fun (arr [], index + 1);
}
```

Output : true.

array मधला element हा 0 पेक्षा लहान असेल तर तो negative असेल. जर negative element सापडला तर true return कर.

find maximum element :-

expectation -: array मधला maximum element find करणे
faith :- बाकीच्या array मधला max element recursion देईल.
base Case = index == arr.length-1 return arr;

```
psvm ( ) {
    int arr [] = {2,4,6,8}
    int val = fun (arr, 0);
    sop (val);
}
int fun ( int arr [], int index) {
    if ( index == arr.length -1) {
            return arr [index];
    }
    int max = fun (arr, index +1);
    if ( arr [index] > max) {
            return arr [index];
    }else {
            return max;
    }
}
```

Output = 8

8) find minimum element :-

expectation - array मधला minimum element शोधायचा.

faith - array मधला minimum element हा recursion देईल.

base Case :- index == arr.length -1 return arr;

```
psvm() {
    int arr [] = { 3, 9, 1, 7 }
    int val = fun ( arr, 0);
    Sop (val);
}
int fun ( int arr [], int index ) {
        if (arr.length == index ) {
                return arr [index];
        }
        int min = fun (arr [], index+1);
        if (arr [index] < min) {
                return arr [index];
        } else {
                return min;
        }
}
```

Output = 1

9) Sum of odd ~~a~~ index : —

expectation — odd index 1,3,5,7 ची sum करायची
faith — बाकीच्या odd index ची sum recursion देईल.
base Case — index > arr.length return 0;

```
psvm ( ){
    int arr [] = { 10,20,30,40,50}
    int val = fun ( arr ,1);
    Sop (val);
}

int fun ( int arr [] , int index){
        if ( ~~arr~~ index > arr.length){
                return 0;
}
~~arr[index] +~~ ~~return~~
~~return fun( arr [] , index+2)~~

    return arr [index] + fun (arr [], index+2);
}
```

output : 60.

1 index = 20 + 3 index वर = 40 20+40 = 60.

1) Second maximum element :-

expectation = array मधून second largest element शोधायचा।

faith = बाकी array मधून recursion second max element काढेल.

base Case = index == arr.length return.

```
psvm ( ){
    int arr [] = { 10, 5, 8, 20, 15 };
    int val = fun ( arr, 0, ~~second~~ 0);
    Sop (val);
}

int fun (int arr, int index, int second){
    if ( index == arr.length){
            return ~~es~~ second;
        }
        if (arr [index] > max) {
            second = max;
            max = arr [index];
        }
        else if ( arr [index] > second) {
            second = arr [index];
        }
    return fun (arr, index+1, second, max);
}
```

O/P = 15.

12) Check Strictly increasing :-

expectation = array strictly increase होतोय का ते बघणे.

faith = बाकीच्या array मधून recursion check करून तो strictly increase होतोय का ते.

base Cond^n = index = arr.length - 1 return true.

```
psvm ( ){
    int arr [ ] = { 1, 2, 3, 3 }
    int val = fun ( arr, 0);
    Sop (val);
}

int fun ( int arr, int index){
        if ( index == arr. length - 1) {
                return true;
        }
        if (arr [index] >= arr [index + 1]){
                return false;
        }
        return fun ( arr, index + 1);
}
```

o/p = true.

Count element greater than x.

expectation : x पेक्षा मोठे element चा count पाहिजे.
faith = बाकी element चा count recursion देईल.
base Case = index == arr.length return 0.

```
psvm ( ) {
    int[]arr = { 1,5,3,7,2 }, x=3.
    int val = fun ( arr, 0, x);
    Sop (val);
}

int fun ( int arr, int index, int x) {
        if (arr.length == index) {
                return 0;
        }
    if ( arr [index] > x) {
            return 1 + fun (arr, index+1, x);
    }else {
            return fun (arr, index+1, x);
}
```

op = 2.

x=2 च्या नंतर 7 & 5 हे 2 च्या no येतात.
त्यामुळे count = 2;

4) Check if array Contain target :

expectation :- target हे array मध्ये आहे का हे check करणे.

faith = next element मध्ये target असेल तर recursion true देईल.

base Case = index == arr.length return false.

```
psvm ( ){
    int arr [] = { 1, 2, 3, 4 }  target = 3
    boolean val = fun ( arr, 0, target);
    Sop (val);
}

    boolean fun ( int arr[] , int index, int target){
        if ( index == arr.length ) {
                return false;
        }
        if (arr [index] == target) {
                return true;
        }
        return fun ( arr, index + 1, target);
}
```

O/P = true.

any एका element आणि target एकच असला तर true return कर.

6] find index of smallest element :-

expectation :- Smallest element चा index return करायचाय.

faith :- बाकी array मध्ये smaller element असेल तर recursion देईल.

base Condⁿ = index == arr.length return index.

```
Psvm () {                                    op:3.
    int arr[] = { 5, 2, 8, 1, 3 }
    int val = fun ( arr, 0, 0 );
    Sop (val);
}
    int fun (int arr, int index, int smallest){
        if ( index == arr.length ) {
                return smallest;
        }
        if (arr [index] < arr [smallest]){
                smallest = index;
        }
        return ~~smallest~~ fun( arr, index+1, smalest);
}
```

o/p = 3.

Smallest element 1 आहे त्याची index 3 आहे ती return केली.

17) Count element divisible by k.

expectations :- k ने divide केलेले element किती आहेत त्याचा count द्य.

faith = बाकी चा count recursion देईल.

base case = arr.length == index return 0.

```
psvm () {
    int arr [] = { 2, 4, 5, 6, 9 } k = 2;
    int val = fun ( arr, 0, k);
    sop (val);
}

int fun ( int arr [], int index, int k){
        if (index == arr.length) {
                return 0;
        }
int count ~~int~~ = fun ( arr, index + 1, k);
        if (arr [index] % k == 0) {
                return 1 + count ;
        }
        return Count;
}
```

Output : 3.

array मधल्या element ला k ने mod केल्यावर जर remainder 0 येत असेल तर count मध्ये add कर. आणि count return कर.

19) Sum of digit of all elements :-

expectation :- प्रत्येक element च्या digit ची sum करणे.

faith :- बाकीच्या element च्या digit ची sum recursion करेल.

base case = index == arr.length return 0;

```
psvm() {
    int arr[]= {12,34,5}
    int val = fun( arr, 0);
    Sop (val);
}

int fun ( int arr[], int index) {
    if ( index == arr.length) {
        return 0;
    }
    int num = arr[index];
    int remainder = 0;
    While ( num > 0) {
        remainder = remainder + num%10;
        num = num/10;
    }
    return remainder + fun (arr, index+1);
}
```

Output : =15

array च्या element चे digit काढले mod करून. त्याची sum केली.

20) Check if array is strictly decreasing :

expectation = array हा strictly decreasing आहे का ते बघणे.

faith = बाकीचा array strictly decrease होतोय का ते recursion सांगेल.

base case = index == arr.length - 1 return true.

```
psvm () {
    int arr [] = {9, 7, 5, 2}
    boolean val = fun (arr, 0);
    Sop (val);
}
boolean fun ( int arr [], int index) {
    if (index == arr.length - 1) {
        return true;
    }
    if (arr [index] <= arr [index + 1]) {
        return false;
    }
    return fun (arr, index + 1);
}
```

Output: true;

जर array मधला element हा दुसरा element पेक्षा लहान असेल तर false return कर.